

ЛАБОРАТОРНА РОБОТА 4

Тема: РЕФАКТОРИНГ КОДУ ТА ПРОВЕДЕННЯ CODE REVIEW ЗА ІНДУСТРІАЛЬНИМИ СТАНДАРТАМИ

Мета роботи:

Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

Код модуля:

<https://github.com/ivannakryzka-a12/casino.github.io/blob/main/betting.js>

Тести:

<https://github.com/ivannakryzka-a12/casino.github.io/blob/main/betting.test.js>

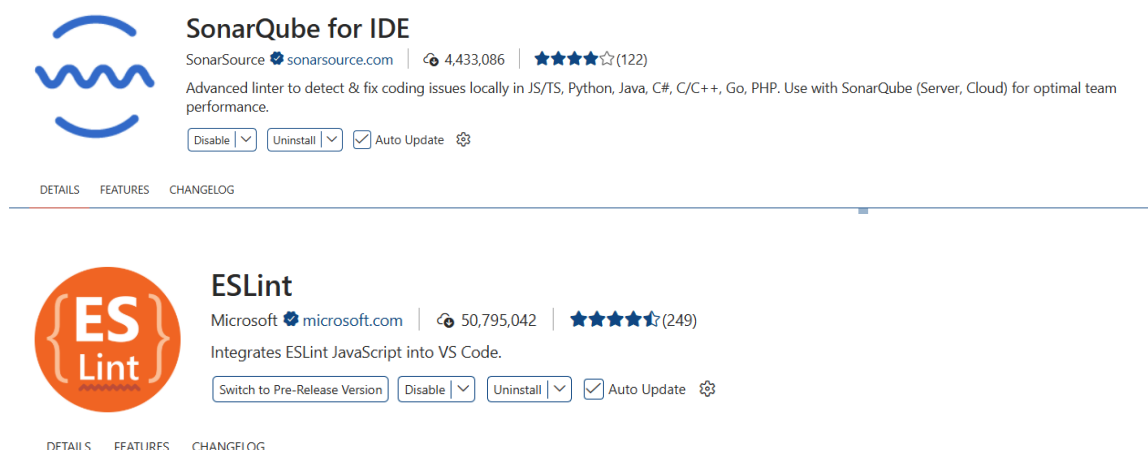
Посилання на гілку:

<https://github.com/ivannakryzka-a12/casino.github.io/tree/review-sofia>

Посилання на Pull Request:

<https://github.com/ivannakryzka-a12/casino.github.io/pull/13>

Встановлення розширення:



The screenshot displays two extensions in the Visual Studio Code interface. The top extension is 'SonarQube for IDE' by SonarSource, which has 4,433,086 downloads and a 4.5-star rating. Its description states it is an advanced linter for detecting and fixing coding issues in various languages. The bottom extension is 'ESLint' by Microsoft, with 50,795,042 downloads and a 4.5-star rating. It is described as integrating ESLint JavaScript into VS Code. Both extension cards include buttons for 'Disable', 'Uninstall', and 'Auto Update', along with a settings gear icon. Navigation tabs for 'DETAILS', 'FEATURES', and 'CHANGELOG' are visible at the bottom of each extension's card.

SonarQube for IDE
SonarSource | 4,433,086 | 4.5 (122)
Advanced linter to detect & fix coding issues locally in JS/TS, Python, Java, C#, C/C++, Go, PHP. Use with SonarQube (Server, Cloud) for optimal team performance.
[Disable] [Uninstall] [Auto Update] ⚙️

ESLint
Microsoft | 50,795,042 | 4.5 (249)
Integrates ESLint JavaScript into VS Code.
[Switch to Pre-Release Version] [Disable] [Uninstall] [Auto Update] ⚙️

ESLint warnings до:

```
PS C:\Users\Сергей\Downloads\casino.github.io> npx eslint betting.js

C:\Users\Сергей\Downloads\casino.github.io\betting.js
  61:1  error  'module' is not defined  no-undef

✖ 1 problem (1 error, 0 warnings)
```

ESLint warnings після:

```
PS C:\Users\Сергей\Downloads\casino.github.io> npx eslint betting.js

C:\Users\Сергей\Downloads\casino.github.io\betting.js
  77:1  error  'module' is not defined  no-undef

✖ 1 problem (1 error, 0 warnings)
```

✓ Should throw error if input is not an array (2 ms)

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	96.55	95.23	100	96.55	
betting.js	96.55	95.23	100	96.55	31

Test Suites: 1 passed, 1 total
Tests: 10 passed, 10 total
Snapshots: 0 total
Time: 1.17 s
Ran all test suites.

Результати Code Review одnogрупника:

№	Рядок коду	Проблема	Категорія	Рекомендація
1	this.VALID_STATUSES = [won, lost]	Масив зі статусами прописаний прямо в конструкторі	Магічні числа	Використати константи з об'єкта BetStatus
2	const isBetInvalid = !bet typeof bet !== object...	Дуже довга і складна умова в одному рядку	Надмірна складність	Розділити перевірки або винести умову в окремий метод
3	Number(total.toFixed(2))	Округлення до конкретної кількості знаків без пояснень	Магічні числа	Замінити двійку на константу DECIMAL_PLACES
4	throw new Error(Invalid input type)	Занадто загальне повідомлення про помилку	Погане іменування	Написати конкретне повідомлення для кожної змінної
5	let total = 0; for (let bet of bets)	Змішування логіки перебору та обчислення	Порушення SRP	Винести логіку підрахунку однієї ставки в окремий метод

Посилання на PR з inline-коментарями Code Review (URL):

<https://github.com/ivannakryzka-a12/casino.github.io/pull/14>

Рефакторинг-операції за результатами отриманого review:

Операція 1

БУЛО: `if (odds <= 1.0)`

СТАЛО: `if (odds <= MIN_ODDS)`

ЧОМУ: Дробове число замінено на іменовану константу. Тепер зрозуміло, що це мінімально допустимий коефіцієнт, а не просто математична перевірка.

Операція 2

БУЛО: `this.VALID_STATUSES = [won, lost]`

СТАЛО: `this.VALID_STATUSES = [BetStatus.WON, BetStatus.LOST]`

ЧОМУ: Текстові значення масиву замінено на використання статусної структури. Це гарантує використання лише дозволених станів без ризику хибного введення.

Операція 3

БУЛО: `return Number(total.toFixed(2))`

СТАЛО: `return Math.round(total * 100) / 100`

ЧОМУ: Усунено складність перетворень типів. Математичне округлення працює прямо з числами і не потребує конвертації змінної у текстовий формат і назад.

Звіт регресійного тестування:

```
● PS C:\Users\Cepрей\Downloads\casino.github.io> npm test

> casino.github.io@1.0.0 test
> jest --coverage

PASS ./betting.test.js
  BettingService Unit Tests
    ✓ Should pass for lower boundary value (3 ms)
    ✓ Should throw error for value below lower boundary (19 ms)
    ✓ Should pass for upper boundary value (2 ms)
    ✓ Should throw error for odds equal to 1.0 (3 ms)
    ✓ Should set status to "won" for valid input (1 ms)
    ✓ Should throw error for invalid status type (3 ms)
    ✓ Should throw error when resolving already finished bet (2 ms)
    ✓ Should correctly calculate sum of all won bets (9 ms)
    ✓ Should ignore lost bets in calculation (6 ms)
    ✓ Should throw error if input is not an array (8 ms)

-----|-----|-----|-----|-----|-----
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----
All files | 96.55   | 95.23    | 100     | 96.55   |
betting.js | 96.55   | 95.23    | 100     | 96.55   | 31
-----|-----|-----|-----|-----|-----
Test Suites: 1 passed, 1 total
Tests:       10 passed, 10 total
Snapshots:   0 total
Time:        0.82 s, estimated 1 s
Ran all test suites.
```

Кількість SonarLint issues до:

```
▼ JS betting.js 5
  ✖ 'module' is not defined. eslint(no-undef) [Ln 77, Col 1]
  ⚠ Don't use a zero fraction in the number. sonarqube(javascript:S7748) [Ln 3, Col 19]
  ⚠ Prefer class field declaration over `this` assignment in constructor for static values. sonarqube(javascript:S7757) [Ln 20, Col 9]
  ⚠ `new Error()` is too unspecific for a type check. Use `new TypeError()` instead. sonarqube(javascript:S7786) [Ln 31, Col 23]
  ⚠ `new Error()` is too unspecific for a type check. Use `new TypeError()` instead. sonarqube(javascript:S7786) [Ln 65, Col 23]
```

Кількість SonarLint issues після:

```
▼ JS betting.js 4
  ⚠ Prefer class field declaration over `this` assignment in constructor for static values. sonarqube(javascript:S7757) [Ln 6, Col 9]
  ⚠ `new Error()` is too unspecific for a type check. Use `new TypeError()` instead. sonarqube(javascript:S7786) [Ln 17, Col 23]
  ⚠ Don't use a zero fraction in the number. sonarqube(javascript:S7748) [Ln 22, Col 22]
  ⚠ `new Error()` is too unspecific for a type check. Use `new TypeError()` instead. sonarqube(javascript:S7786) [Ln 49, Col 23]
```

Цикломатична складність до:

```
PS C:\Users\Cepрей\Downloads\casino.github.io> python -m lizard betting.js
=====
NLOC    CCN    token  PARAM  length  location
-----
      5      1     26      0       5 constructor@5-9@betting.js
     12      6     79      2      12 placeBet@15-26@betting.js
     10      5     62      2      10 resolveBet@32-41@betting.js
     12      4     71      1      12 calculateTotalPayout@47-58@betting.js
1 file analyzed.
=====
NLOC    Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
     42       9.8    4.0      59.5          4  betting.js

=====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100)
=====
Total nloc  Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
     42       9.8    4.0      59.5      4          0    0.00  0.00

PS C:\Users\Cepрей\Downloads\casino.github.io>
```

Цикломатична складність після:

```
PS C:\Users\Cepрей\Downloads\casino.github.io> python -m lizard betting.js
=====
NLOC    CCN    token  PARAM  length  location
-----
      5      1     26      0       5 constructor@19-23@betting.js
     12      6     79      2      12 placeBet@29-40@betting.js
     10      5     70      2      12 resolveBet@46-57@betting.js
     12      4     73      1      12 calculateTotalPayout@63-74@betting.js
1 file analyzed.
=====
NLOC    Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
     51       9.8    4.0      62.0          4  betting.js

=====
No thresholds exceeded (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100)
=====
Total nloc  Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
     51       9.8    4.0      62.0      4          0    0.00  0.00
```

Метрика	До	Після
Цикломатична складність (max)	6 (placebet)	6 (placebet)
SonarLint issues	5	4
ESLint warnings	0	0
Тести (pass / total)	10 / 10	10 / 10

Підсумкова рефлексія:

Роль рецензента дозволила мені поглянути на процес розробки з іншої сторони та навчитися об'єктивно й критично оцінювати чужі технічні рішення. Найціннішим практичним досвідом став детальний аналіз структури функцій для виявлення надмірної складності умов та порушень базових принципів проєктування, зокрема принципу єдиної відповідальності (SRP). Я переконалася, що написання самоописового та зрозумілого коду за індустріальними стандартами є таким же важливим етапом, як і забезпечення його базової працездатності. Крім того, такий формат взаємодії значно покращив наші навички професійної комунікації всередині команди розробників.